UMHACKATHON 2026 · DOMAIN 1 · AI SYSTEMS & AGENTIC WORKFLOW AUTOMATION

Product Requirements.

The product the platform is, the users it serves, the scope it commits to for the UMHackathon 2026 submission.

TEAM · OPUS MAGNUM

Oybek Odilov
Al-Bazeli Gameel Abduljalil
Lee Yih Shen
Samandar Erkinjonov
Abdugaffor Odilov

ONBOARD

POWERED BY Z.AI GLM · REGULATED ONBOARDING ENGINE

PRD · PRODUCT REQUIREMENTS APRIL 2026 SUBMISSION RELEASE

1. Problem Definition & Purpose

Opening a new building is slow, paper-heavy, and local. Ownership certificates, fire-safety approvals, zoning clearances, floor plans, safety reports — each submitted to a different office, each with its own format, each city wanting it slightly differently. Developers file paperwork they do not fully understand and wait weeks for feedback; cities get a flood of non-standard submissions; the public has no good way to discover what is new, approved, or available.

The bottleneck is not bureaucracy as a concept. It is the **absence of a system that can read what's submitted, check it against what a city requires, and keep the process moving without waiting on humans to re-read every form**.

Purpose. Opus Magnum turns regulated building onboarding into a single guided workflow, powered end-to-end by an AI that reads documents, verifies them against city-specific criteria, reconciles contradictions, and publishes the building as a public, explorable listing.

Larger ambition. Buildings are the concrete demo. The engine underneath is a **general-purpose adaptive, multi-modal, AI-verified onboarding platform** — the same template model applies to supplier qualification, license renewal, KYC, regulated-professional registration. We ship buildings because the pain is concrete and visual; the pattern is reusable.

2. Target Users & User Stories

Three roles, one engine, one knowledge base.

2.1 Admin — the city's configuration point

Who. City-office workflow owner (municipal digital-services manager, or equivalent). **Core job.** Define the onboarding process a city requires: which forms, which documents, which verification criteria, in what order.

Stories.

- *As an admin, I can describe my city's requirements in one sentence and get a GLM-drafted starter template I can edit, so I don't start from a blank page.*
- *As an admin, I can ask a tutor assistant to explain why each step exists and what a field should capture, so I can author confidently without a manual.*
- *As an admin, I can publish a template and have every new onboarding in my city run through that exact sequence, so that submissions become comparable across buildings.*

2.2 Building owner — the workflow user

Who. Developer, property-management lead, or their appointed submitter. **Core job.** Onboard a specific building by running that city's template to completion.

Stories.

- *As an owner, I can see the whole onboarding process as a live graph, so I always know what is done, what is running, and what is waiting on me.*
- *As an owner, when I upload a compliance document, I can watch the AI process it in real time — received, parsed, extracted, verified — and see a plain-language pass or fail verdict with evidence.*
- *As an owner, when the AI flags a contradiction between documents, I can see exactly which values disagree and correct them without a round-trip to a reviewer.*
- *As an owner, when all steps pass, my building is published automatically — the paperwork and the public listing come from the same submission.*

2.3 Visitor — the public audience

Who. Anyone curious about a published building — prospective tenants, journalists, city residents, neighbouring businesses. **Core job.** Discover a building and ask grounded questions about it.

Stories.

- *As a visitor, I can browse a directory of published buildings and open one to see a 3D view and a chat panel.*
- *As a visitor, I can ask the assistant anything about the building, and it answers only from what the owner submitted — with citations to the source document or profile field, never invention.*

3. Originality, Innovation & Value Realisation

3.1 What is original

The workflow is data, not code. A new city is a new JSON template stored in Postgres. The same engine runs every template. There is no per-city codebase. No YAML rule files. No "compliance-rules microservice." The admin's template *is* the rule set.

Compliance docs vs content docs are explicit first-class types. Compliance docs feed pass/fail verdicts and are never shown publicly. Content docs feed the building's profile and ground the visitor assistant. The same upload workflow handles both, the system never conflates them, and the owner submits everything once.

Every GLM call is logged for audit. Each step's `decision_log` captures the prompt, the tool called, the model's arguments, and the duration. This is the drift net for AI behaviour and the substrate for future human-in-the-loop review of AI judgements.

3.2 Why it matters

- **For admins:** a city that wants to onboard buildings for the first time can be live in minutes, not quarters. The marginal cost of a new jurisdiction is one template.
- **For owners:** the feedback loop collapses from weeks to seconds. A rejection cites the exact reason; corrections go straight back into the graph.
- **For cities:** submissions become uniform and auditable. Every decision trail is preserved.
- **For the public:** discovery becomes first-class. A building's page is not a document dump — it is a conversable, grounded listing.

3.3 The "AI at the centre" invariant

If GLM is removed, five things stop working:

1. **Template drafting.** Admins lose the GLM-generated starter template and the tutor assistant; only manual authoring remains.
2. **Document parsing & fact extraction.** Raw PDFs / images never become structured fields.
3. **Compliance verification.** Plain-English criteria cannot be applied to a document without reading it.
4. **Cross-document reconciliation.** Contradiction surfacing requires reasoning over multiple extracted outputs.
5. **Visitor assistant.** Every grounded answer depends on reading the building's content documents.

Form rendering survives. Nothing else does. This is the architectural litmus test judged in §6.

4. Feature Prioritisation & MVP Scope

4.1 In scope for MVP

AREA	FEATURE	STATUS
Admin	City CRUD	Shipped
Admin	Template editor with primitive-specific config	Shipped
Admin	`draft_with_ai` (GLM drafts full template from one-line brief)	Shipped
Admin	Template assistant chat (SSE-streamed, explains fields)	Shipped
Owner	Building create + city selection	Shipped
Owner	Workflow canvas (React Flow) with live step status	Shipped
Owner	Dynamic form renderer for `collect_form`	Shipped
Owner	Drag-drop upload with live per-file processing cards	Shipped
Owner	Compliance verification loop (re-upload on fail)	Shipped
Owner	Content extraction into the building profile	Shipped
Owner	Cross-document check with severity grading	Shipped
Owner	Human-review summary + publish	Shipped
Owner	Per-step decision-log drawer (every GLM call visible)	Shipped
Visitor	Published directory	Shipped
Visitor	3D scene (real GLB apartment models + scene_config overlay)	Shipped
Visitor	Grounded assistant chat (SSE-streamed, profile + content docs)	Shipped
Platform	Six workflow primitives, composable per city	Shipped
Platform	LangGraph StateGraph per run with Postgres/SQLite checkpointer	Shipped
Platform	OpenAI-compatible integration with ILMU `ilmu-glm-5.1`	Shipped
Platform	`decision_log` persisted per step	Shipped

4.2 Out of scope — flagged explicitly

- **Real auth.** Mocked session: one seeded admin and one seeded owner. Drop-in JWT is the designed upgrade path; this was cut to focus engineering on the GLM-critical loop.
- **Payments / transactions.** Not in the onboarding value chain for MVP.
- **Multi-tenant isolation & row-level security.** Single-tenant demo.
- **Production observability & alerting.** Dev-grade logging only.
- **Second jurisdiction.** The engine is config-driven; only Shah Alam is authored for the submission. Adding Kuala Lumpur would be one JSON file.
- **Vision documents.** `ilmu-glm-5.1` does not accept image inputs (verified empirically — see §7 Risks). Image documents go through OCR (`pytesseract`) before GLM sees them.

4.3 Prioritisation rationale

The demo judged here is three minutes long. Every shipped feature earns a beat in that demo. Features that do not appear in the demo script (real auth, payments, observability) were cut so the GLM-critical path could be built deeply rather than broadly.

5. AI Model & Prompt Design

5.1 Model selection

The platform uses **`ilmu-glm-5.1`** through the ILMU API (`https://api.ilmu.ai/v1`), which is OpenAI-compatible and surfaces tool-use, JSON mode, and SSE streaming through the standard OpenAI Python SDK. Three properties make it the right fit for an AI-centric onboarding engine:

- **Long context (200k tokens).** Whole content-document corpora for a single building fit inline, so the visitor assistant can ground answers without an embeddings layer for the MVP.
- **Native tool use + structured output.** Every primitive that needs reasoning (`extract_document` , `verify_against_criteria` , `cross_check_documents` , `summarize_for_review` , `draft_template` , `explain_field` , `answer_visitor_question`) is exposed as a typed function and returns parsed arguments, eliminating brittle text parsing.
- **Reasoning-effort knob.** `reasoning_effort="low"` is materially faster for extraction and verification, while free-form tutor and visitor streams omit the parameter to let the model think when grounding requires it.

Using any other reasoning model is explicitly disqualifying for this domain; GLM is therefore both a product requirement and a hackathon constraint.

5.2 Prompting strategy

The platform mixes two distinct strategies, chosen per surface:

- **Few-shot, schema-forced tool calls** for compliance and content primitives. Every reasoning step issues a single `chat.completions.create` with `tool_choice="required"` against a tightly-scoped tool schema (`extract_document` , `verify_against_criteria` , etc.) and a system prompt that names the criteria, the doc class, and the success contract. This eliminates hallucinated structures and keeps every call auditable.
- **Multi-step agentic prompting** for the human-facing assistants. The admin tutor and visitor chat both run as SSE-streamed dialogues with the conversation history and a domain-grounding payload (template steps for the tutor; building profile + content-doc extracts for the visitor). The system prompt enforces "answer only from the grounding material; otherwise say you do not know."

We chose this split because the verification path needs *deterministic shape* (parsed JSON arguments) while the tutoring path needs *open-ended reasoning over rich context*. Mixing the two would make verification fragile and the tutor robotic.

5.3 Context and input handling

- **Compliance documents** are streamed through `processing/pipeline.py` : `pypdf` extracts the text layer, falling back to `pdf2image + pytesseract` OCR if the layer is empty. Images route directly to `pytesseract` . GLM never sees raw PDFs because `ilmu-glm-5.1` does not currently accept image inputs (verified empirically).
- **Per-document budget.** Extracted text is truncated to **~30,000 characters** before reaching `extract_document` ; verification works on the structured output, not the raw text, so this ceiling never starves the criteria check.
- **Visitor grounding budget.** The visitor assistant assembles `building.profile + scene_config + content-doc extracts` and caps the grounding payload at **≤ 60,000 characters** (well inside GLM's 200k window). Anything longer is chunked oldest-first into a "context summary" block.
- **Hard rejections.** Empty extractions (OCR returned 0 chars) are surfaced as "document unreadable — please re-upload as searchable PDF" before any GLM call is made. We never feed empty payloads to the model.

5.4 Fallback and failure behavior

- **Transient API errors (429 / 5xx).** `GLMClient._chat` retries up to 4 times with exponential backoff and jitter.
- **Malformed tool arguments.** `call_tool` catches `json.JSONDecodeError` and returns `{_error, _raw}` so the primitive marks the step `failed` with a retry affordance instead of crashing the run.
- **Hallucinated or off-topic completions.** Verification primitives compare structured arguments against the criteria list — a "verdict": "pass" with no cited evidence is rejected and re-prompted once; on a second failure the step is marked `failed` for human review.
- **Visitor grounding miss.** The system prompt forces a "I don't know based on the documents this owner submitted" response when the grounding material lacks the answer. We do not fall back to general knowledge.
- **Human escalation path.** Every primitive can route to `human_review`, which surfaces the failing step's `decision_log` and a GLM-written summary so a reviewer can override or restart.
- **Auditability.** Every call — successful or failed — writes to `step_runs.decision_log` with prompt, tool, arguments, response, and duration. A failure that escapes review is forensically reproducible.

6. Success Metrics (MVP)

METRIC	TARGET	HOW MEASURED
Live GLM calls per run	≥ 12 per full Shah Alam template	`decision_log` count per `workflow_run`
Compliance verification accuracy	≥ 9 / 10 on labelled fixtures	`tests/test_upload_compliance.py` + GLM eval set
Verification rejection with evidence	100% of failing cases cite offending text	Decision-log inspection
End-to-end demo time	≤ 3 minutes live	Timed rehearsal
Template authoring time	≤ 2 minutes from blank to publish via `draft_with_ai`	Manual measurement

7. GLM-Centrality Validation

Every one of the five critical surfaces calls GLM at runtime, each logged under `step_runs.decision_log` with its tool name, arguments, and timing. A reviewer who stubs `GLMClient` to raise immediately would see the workflow freeze at step 1, template drafting fail, and the visitor assistant return an error. The litmus test passes.

8. Assumptions & Constraints

8.1 LLM cost constraint

We estimate **~15–25 GLM calls per full Shah Alam onboarding** (one per `extract_document` per uploaded doc, one per `verify_against_criteria`, plus admin and visitor turns). With `ilmu-glm-5.1`'s typical structured-output sizing and `reasoning_effort="low"` on extraction/verification, an average end-to-end run lands around **40–80k input + ~6–10k output tokens**.

Two design decisions hold the cost line:

- **`reasoning_effort="low"` is the default on every structured-output path.** Extraction and verification do not benefit from deep reasoning; the model still emits the right schema, faster and cheaper.
- **Decision-log content elision.** Long message bodies in the decision log are truncated past 4k characters so we never re-feed entire documents back into prompts during retries or human review.

A future cost lever (not in MVP scope) is response caching of `extract_document` keyed by file SHA — a re-uploaded duplicate would skip the model entirely.

8.2 Technical constraints

- **Vision unavailable on `ilmu-glm-5.1`.** Image documents must pass through OCR before the model sees them. PDFs without a text layer use the `pdf2image + pytesseract` fallback; raw images go directly to `pytesseract`.
- **Auth is mocked.** A seeded admin and a seeded owner stand in for a real identity provider. Drop-in JWT is the designed upgrade path.
- **Single-tenant demo.** No row-level security or per-tenant data partitioning.
- **One jurisdiction is authored.** The engine accepts any number of city templates; the submission ships Shah Alam end-to-end. A second city is one JSON file.

8.3 Performance constraints

- **GLM rate limits.** ILMU's published rate limits apply; the demo path has been profiled to stay below the per-minute quota at one concurrent run.
- **OCR throughput.** `pdf2image + pytesseract` on a multi-page scanned PDF can take 5–15 seconds; users see a per-file processing card with live status so latency is never silent.
- **SSE polling cadence.** The run-progress SSE endpoint polls the DB at 500 ms — good for one demo run, replaceable by a Redis pub/sub bus for production load.

8.4 User input

- **Owner-submitted documents are trusted as-is.** The system never modifies an uploaded file; it only extracts, verifies, and audits.
- **Required-field gating.** A `collect_form` step cannot advance until the owner submits the configured fields; the React Flow canvas highlights the awaiting node until input arrives.
- **Re-upload loop on failure.** A failed verification re-routes to the same upload step rather than crashing the run; `pending_input[step_id]` is cleared so the new upload kicks off a fresh GLM verification.
- **Human review is the final stop.** When `human_review` fires, no automated decision overrides the reviewer; the model's summary is advisory, not authoritative.

9. Risks & Mitigations

RISK	IMPACT	MITIGATION
`ilmu-glm-5.1` rate limits trigger during demo	Mid-demo stall	Exponential-backoff retry in `GLMClient._chat`; demo path uses ≤ 15 calls end-to-end
Vision not supported on GLM 5.1	Image docs can't be read directly	OCR-only fallback (`pytesseract`), flagged in decision log; baseline verified
GLM returns malformed tool arguments	Step stalls	`call_tool` catches `json.JSONDecodeError`, returns `{_error}` so the UI can show a retry affordance
Owner submits a scanned PDF with no text layer	Silent empty extract	`pdf.py` falls back to `pdf2image` + `pytesseract`; decision log marks `ocr_used=true`
Template becomes invalid mid-edit (unknown primitive)	Run crashes	`compile_template` raises `ValueError("unknown primitive: …")` before the graph executes
Checkpointer DB corruption	Run cannot resume	SQLite file scoped to `var/langgraph.sqlite`; deletable without data loss (state also mirrored in `workflow_runs.state`)

10. Future Work (not in this submission)

- Second city authored (Kuala Lumpur) to prove config-only expansion.
- Real multi-tenant auth + per-seat isolation.
- Embeddings-based RAG for very large content corpora (fallback if single-building content exceeds 200k tokens).
- Vision-capable model upgrade when ILMU adds one to our plan.
- City-side reviewer surface that spot-checks GLM verdicts.

Source: `implementation.md` §1–19 · `idea-presentation.md` · this document is the canonical product statement for the submission.